

## บทที่ 5 แพตเทิร์นพฤติกรรม (Behavioral Patterns)

### 5.1 ความสำคัญของแพตเทิร์นพฤติกรรม

แพตเทิร์นพฤติกรรม (Behavioral Patterns) เป็นกลุ่มแพตเทิร์นที่ช่วยแก้ปัญหาเกี่ยวกับพฤติกรรมของอ็อบเจกต์ (object) ที่มีต่อกัน โดยเน้นการจัดการพฤติกรรม และปฏิสัมพันธ์ระหว่างอ็อบเจกต์ในระบบ

**พฤติกรรมของอ็อบเจกต์** คือ "สิ่งที่อ็อบเจกต์ทำ" เป็นฟังก์ชันการทำงาน หรือการดำเนินการ (Operation) ที่อ็อบเจกต์มี พฤติกรรม ได้แก่

- 1) เมธอด (Method) หรือฟังก์ชัน เป็นตัวแทนโดยตรงของพฤติกรรม เป็นโค้ดที่ระบุขั้นตอนการทำงานเมื่อถูกเรียกใช้ เช่น อ็อบเจกต์ Car มีเมธอด `drive()` `break()` `openFrontLight()`
- 2) การโต้ตอบกับอ็อบเจกต์อื่น เป็นวิธีการที่อ็อบเจกต์หนึ่งส่งข้อความ (Message) หรือเรียกใช้เมธอดของอ็อบเจกต์อื่น เพื่อให้เกิดการทำงานร่วมกัน เช่น อ็อบเจกต์ Driver สั่งให้อ็อบเจกต์ Car ทำการ `move()`
- 3) การเปลี่ยนแปลงสถานะ (State) พฤติกรรมส่งผลให้สถานะ ได้แก่ ข้อมูลภายใน หรือ Attributes ของอ็อบเจกต์เปลี่ยนแปลงไป เช่น เมื่ออ็อบเจกต์ Account ใช้เมธอด `withdraw(amount)` สถานะ `balance` ก็จะลดลง

การทำความเข้าใจพฤติกรรมของอ็อบเจกต์เป็นสิ่งสำคัญอย่างยิ่ง เพราะรูปแบบเหล่านี้ถูกออกแบบมาเพื่อ

- จัดระเบียบ และกำหนดความรับผิดชอบ (Responsibility) ของพฤติกรรม ทำให้แต่ละอ็อบเจกต์มีหน้าที่ชัดเจน ไม่ซ้ำซ้อน
- กำหนดรูปแบบการสื่อสาร (Communication) ระหว่างอ็อบเจกต์ เช่น การส่งต่อคำสั่ง (Command Pattern) หรือการแจ้งเตือนเมื่อมีการเปลี่ยนแปลง (Observer Pattern)
- ทำให้พฤติกรรมสามารถเปลี่ยนแปลง และนำกลับมาใช้ใหม่ได้ง่าย (Flexibility and Reusability) เช่น การแยกอัลกอริทึมออกมาเป็นอ็อบเจกต์ต่างหากเพื่อให้สามารถเปลี่ยนพฤติกรรมได้ง่ายใน Runtime (Strategy Pattern)

แพตเทิร์นพฤติกรรม มีประโยชน์หลายด้านดังต่อไปนี้

- 1) ลดความซับซ้อนของการสื่อสารระหว่างวัตถุ โดยกำหนดวิธีการที่วัตถุจะสื่อสารกันอย่างมีประสิทธิภาพ และลดการพึ่งพา (dependency) ระหว่างวัตถุ ตัวอย่างเช่น Mediator Pattern ช่วยรวมศูนย์การสื่อสารระหว่างวัตถุหลาย ๆ ตัว ผ่านตัวกลาง (mediator) แทนที่จะเชื่อมโยงโดยตรง ทำให้โค้ดมีความยืดหยุ่นมากขึ้น
- 2) เพิ่มความสามารถในการขยายระบบ ทำให้ระบบสามารถเพิ่มหรือเปลี่ยนแปลงพฤติกรรมของโปรแกรมโดยไม่ต้องแก้ไขโค้ดเดิม ตัวอย่างเช่น Strategy Pattern ช่วยแยกการตัดสินใจเชิงตรรกะออกจากส่วนอื่น ๆ ของระบบ ทำให้สามารถเพิ่มกลยุทธ์ใหม่ได้ง่าย
- 3) ส่งเสริมการใช้โครงสร้างที่ยืดหยุ่น ทำให้ระบบมีโครงสร้างที่รองรับการเปลี่ยนแปลงในอนาคต ตัวอย่างเช่น Observer Pattern ทำให้วัตถุหนึ่งสามารถแจ้งการเปลี่ยนแปลงไปยังวัตถุอื่นได้โดยอัตโนมัติ
- 4) สนับสนุนการพัฒนาแบบ Agile โดยทำให้การปรับปรุงหรือเปลี่ยนแปลงพฤติกรรมของระบบง่ายขึ้น เนื่องจากมีการแยกความรับผิดชอบ (separation of concerns) อย่างชัดเจน
- 5) ปรับปรุงความสามารถในการทดสอบ ช่วยให้แยกส่วนในการทดสอบได้ง่าย เช่น Command Pattern จะ encapsulate action ต่าง ๆ ไว้ในวัตถุ ทำให้สามารถทดสอบ action เหล่านี้ได้แยกส่วน

แพตเทิร์นพฤติกรรม มีทั้งหมด 11 แพตเทิร์น ดังรายละเอียดในหัวข้อต่อไปนี้

## 5.2 Command Pattern

Command Pattern เป็นแพตเทิร์นพฤติกรรมที่ใช้สำหรับห่อหุ้ม (encapsulate) คำสั่ง (command) ให้อยู่ในรูปของอ็อบเจกต์ (object) โดยแยกคำสั่งออกจากผู้ส่งคำสั่ง (invoker) และผู้ปฏิบัติงาน (receiver) ทำให้สามารถจัดการ เก็บประวัติ ยกเลิก หรือเรียกซ้ำได้

### :: *Command Pattern* ::

ห่อหุ้ม (encapsulate) คำสั่ง (command) ให้อยู่ในรูปของอ็อบเจกต์ (object)  
โดยแยกคำสั่ง (command) ออกจากผู้ส่งคำสั่ง (invoker) และผู้ปฏิบัติงาน (receiver)

### ตัวอย่างปัญหา

บ้านอัจฉริยะที่ควบคุมอุปกรณ์ เช่น ไฟในห้อง ประตูอัตโนมัติ เครื่องปรับอากาศ ผู้ใช้สามารถส่งคำสั่งเปิด/ปิดไฟ ล็อก/ปลดล็อกประตู เปิด/ปิดเครื่องปรับอากาศ

### ตัวอย่าง Main program

```
// สร้าง Receiver
Light livingRoomLight = new Light();

// สร้าง Commands
Command turnOnLight = new TurnOnLightCommand(livingRoomLight);
Command turnOffLight = new TurnOffLightCommand(livingRoomLight);

// สร้าง Invoker
RemoteControl remote = new RemoteControl();

// การควบคุม
System.out.println("== Control Light ==");
remote.setCommand(turnOnLight);
remote.pressButton(); // เปิดไฟ
remote.pressUndo();   // ปิดไฟ
```

## 5.2.1 วัตถุประสงค์และตัวอย่าง

วัตถุประสงค์หลักของ Command Pattern คือการห่อหุ้ม (encapsulate) คำขอ (request) หรือการดำเนินการ (operation) ให้เป็นอ็อบเจกต์ นั่นคือการแปลงคำขอ เช่น "เปิดไฟ" หรือ "บันทึกไฟล์" ให้เป็นอ็อบเจกต์เดียวที่เรียกว่า Command ได้แก่ OpenLightCommand หรือ WriteFileCommand

ในบริบทของบ้านอัจฉริยะ (Smart Home) มีหลายสถานการณ์ที่ Command Pattern ช่วยให้ระบบมีความยืดหยุ่นและขยายได้ง่าย เช่น

- การควบคุมอุปกรณ์หลายชนิด ไฟ ประตู เครื่องปรับอากาศ หรือม่านหน้าต่าง คำสั่งสำหรับเปิด (OpenCommand) ปิด (CloseCommand) หรือเปลี่ยนสถานะ (ChangeStatusCommand)
- การเก็บประวัติคำสั่ง บันทึกคำสั่งที่ดำเนินการไปแล้ว เพื่อรองรับการ undo/redo เนื่องจากคำสั่งถูกเก็บไว้เป็นอ็อบเจกต์ ทำให้สามารถเก็บประวัติการทำงานไว้ในโครงสร้างข้อมูล เช่น Stack เพื่อย้อนกลับการดำเนินการที่ผ่านมาได้ง่าย

- การจัดคิว/การบันทึก (Queueing/Logging) สามารถจัดเก็บชุดคำสั่งไว้ในคิว (Queue) เพื่อให้รันในภายหลัง เช่น การทำงานแบบ Batch หรือบันทึกคำสั่งเพื่อนำมาเล่นซ้ำ (Replay) ในกรณีที่ระบบล้มเหลว (Transaction Logging)
- การจัดโครงสร้างความสัมพันธ์ในระบบที่ซับซ้อน เช่น Macro (ชุดคำสั่งที่ถูกรวมไว้ในคำสั่งเดียว) การสร้างเมนู หรือปุ่มกดที่สามารถเชื่อมต่อการดำเนินการต่าง ๆ ได้อย่างยืดหยุ่น

### ตัวอย่างการใช้งาน

บ้านอัจฉริยะที่ควบคุมอุปกรณ์ เช่น ไฟในห้อง ประตูอัตโนมัติ เครื่องปรับอากาศ ผู้ใช้สามารถส่งคำสั่ง

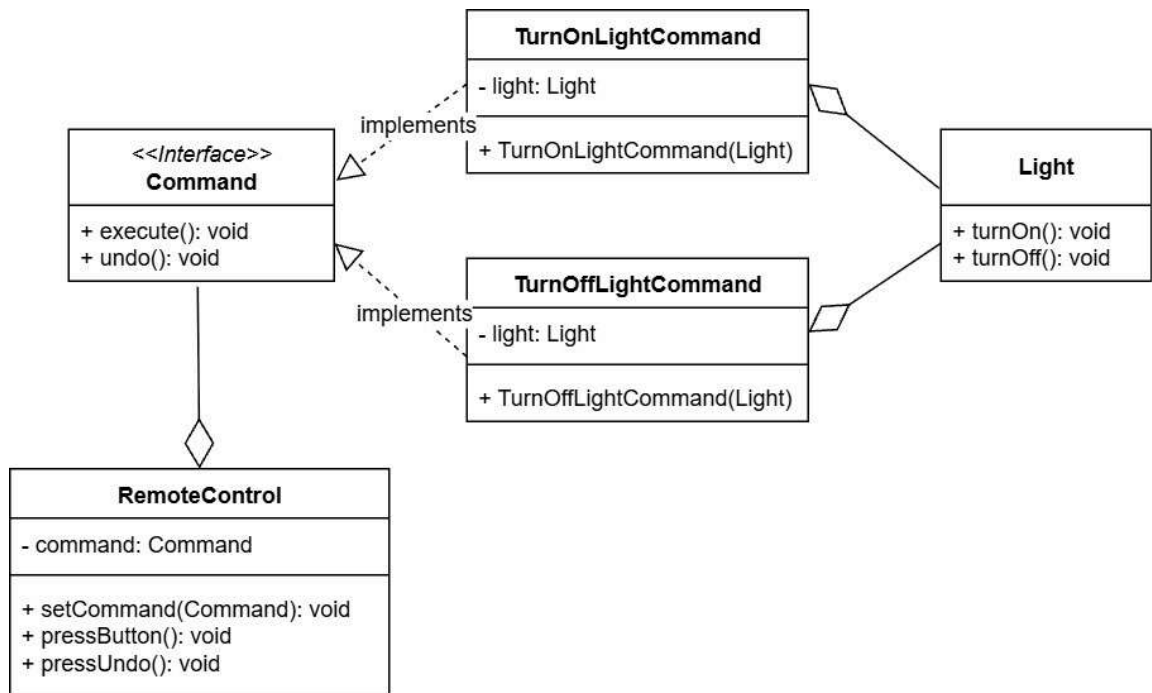
- เปิด/ปิดไฟ
- ล็อก/ปลดล็อกประตู
- เปิด/ปิดเครื่องปรับอากาศ

เมื่อกดปุ่ม → คำสั่งจะถูกดำเนินการ

เมื่อกดปุ่ม undo → ย้อนกลับคำสั่งที่ทำไว้

### โครงสร้าง Command Pattern

- Command Interface กำหนดคำสั่งที่สามารถทำงานได้ เช่น execute และ undo
- Concrete Command คำสั่งเฉพาะ เช่น เปิด/ปิดไฟ ล็อก/ปลดล็อกประตู
- Receiver อุปกรณ์ที่ปฏิบัติตามคำสั่ง
- Invoker ตัวที่ส่งคำสั่ง เช่น รีโมทหรือแอปพลิเคชัน
- Client ตัวที่กำหนดการทำงานของระบบ



ภาพที่ xx บ้านอัจฉริยะที่ควบคุมอุปกรณ์ โดยใช้ Command Pattern

**Command Interface** กำหนดคำสั่งที่สามารถทำงานได้ คือ execute

```

1. // Command Interface
2. interface Command {
3.     void execute();
4.     void undo();
5. }
  
```

สร้าง **Receiver** เช่น Light และ Door คืออุปกรณ์ที่ทำหน้าที่ปฏิบัติตามคำสั่ง

```

1. // Receiver: อุปกรณ์ไฟ
2. class Light {
3.
4.     public void turnOn() {
5.         System.out.println("The light is ON");
6.     }
7.
8.     public void turnOff() {
9.         System.out.println("The light is OFF");
10.    }
11.}
  
```

สร้าง **Commands** TurnOnLightCommand เป็นคำสั่งที่ใช้ควบคุม Receiver

```
1. // Concrete Command: เปิดไฟ
2. class TurnOnLightCommand implements Command {
3.
4.     private Light light;
5.
6.     public TurnOnLightCommand(Light light) {
7.         this.light = light;
8.     }
9.
10.    @Override
11.    public void execute() {
12.        light.turnOn();
13.    }
14.
15.    @Override
16.    public void undo() {
17.        light.turnOff();
18.    }
19.}
```

```
1. // Concrete Command: ปิดไฟ
2. class TurnOffLightCommand implements Command {
3.
4.     private Light light;
5.
6.     public TurnOffLightCommand(Light light) {
7.         this.light = light;
8.     }
9.
10.    @Override
11.    public void execute() {
12.        light.turnOff();
13.    }
14.    @Override
15.    public void undo() {
16.        light.turnOn();
17.    }
18.}
```

**Invoker** RemoteControl รับคำสั่งจากผู้ใช้ เช่น กดปุ่ม และเรียก execute() หรือ undo()

```
1. // Invoker: รีโมตคอนโทรล
2. class RemoteControl {
3.
4.     private Command command;
5.
6.     public void setCommand(Command command) {
7.         this.command = command;
8.     }
9.
10.    public void pressButton() {
```

```

4.         command.execute();
5.     }
6.
7.     public void pressUndo() {
8.         command.undo();
9.     }
10.}

```

การใช้งานใน main program

```

1. // Main Class
2. public class SmartHome {
3.     public static void main(String[] args) {
4.
5.         // สร้าง Receiver
6.         Light livingRoomLight = new Light();
7.         Door frontDoor = new Door();
8.
9.         // สร้าง Commands
10.        Command turnOnLight = new
11.            TurnOnLightCommand(livingRoomLight);
12.        Command turnOffLight = new
13.            TurnOffLightCommand(livingRoomLight);
14.
15.        // สร้าง Invoker
16.        RemoteControl remote = new RemoteControl();
17.
18.        // การควบคุม
19.        System.out.println("== Control Light ==");
20.        remote.setCommand(turnOnLight);
21.        remote.pressButton(); // เปิดไฟ
22.        remote.pressUndo();   // ปิดไฟ
23.
24.    }
25.}

```

ผลลัพธ์

== Control Light ==

The light is ON

The light is OFF

ประโยชน์ของการใช้ Command Pattern

- การเพิ่มความยืดหยุ่น เพิ่มคำสั่งใหม่ เช่น “เปิดเครื่องปรับอากาศ” ได้ง่าย
- รองรับการทำ Undo/Redo รองรับการย้อนกลับคำสั่งหรือทำคำสั่งซ้ำ

- ลดการพึ่งพา แยกผู้ส่งคำสั่ง (Invoker) ออกจากผู้ปฏิบัติงาน (Receiver)
- เพิ่มความสามารถในการบันทึกประวัติคำสั่ง ใช้สำหรับสร้างระบบ automation หรือ history ในระบบ Smart Home

### 5.2.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Command Pattern เหมาะสำหรับสถานการณ์ที่ต้องการความยืดหยุ่นในการจัดการคำสั่ง และรองรับการปรับเปลี่ยน/เพิ่มคำสั่งใหม่โดยไม่กระทบโครงสร้างเดิม เช่น การเก็บประวัติ (history) การยกเลิก (undo) หรือการทำซ้ำ (redo) ตัวอย่างสถานการณ์จริงที่ใช้ Command Pattern มีดังนี้

**ตัวอย่างที่ 1 ระบบจัดการคำสั่งในโปรแกรมกราฟิก** ในโปรแกรมออกแบบกราฟิก เช่น Photoshop หรือ Figma ผู้ใช้ต้องการทำงานกับคำสั่งที่หลากหลาย เช่น วาดเส้น เปลี่ยนสี ลบวัตถุ และต้องรองรับการ undo/redo หรือบันทึกลำดับคำสั่งเพื่อทำซ้ำ

การใช้ Command Pattern

- สร้างคำสั่ง เช่น DrawLineCommand, ChangeColorCommand, และ DeleteObjectCommand
- เก็บคำสั่งใน stack เพื่อรองรับ undo/redo
- รองรับการบันทึกคำสั่งสำหรับ macro (การทำงานซ้ำอัตโนมัติ)

โครงสร้างของระบบ

- Command Interface กำหนดคำสั่ง execute() และ undo()
- Concrete Command คำสั่งเฉพาะ เช่น DrawCommand, DeleteCommand
- Receiver ส่วนที่ทำหน้าที่ประมวลผลคำสั่ง เช่น Canvas
- Invoker ส่วนที่รับคำสั่งจากผู้ใช้และจัดการกับคำสั่ง เช่น การเรียก execute หรือ undo
- Client ส่วนที่กำหนดการทำงานของระบบ

Interface Command

```
1. // Command Interface
2. interface Command {
3.     void execute();
4.     void undo();
5. }
```

Receiver (Canvas) รับคำสั่งสำหรับการวาดและลบรูปทรง

```
1. // Receiver: Canvas
2. class Canvas {
3.
4.     public void draw(String shape) {
5.         System.out.println("Drawing: " + shape);
6.     }
7.
8.     public void erase(String shape) {
9.         System.out.println("Erasing: " + shape);
10.    }
11.}
```

Command DrawCommand คำสั่งสำหรับวาดรูป EraseCommand คำสั่งสำหรับลบรูป

```
1. // Concrete Command: Draw Command
2. class DrawCommand implements Command {
3.
4.     private Canvas canvas;
5.     private String shape;
6.
7.     public DrawCommand(Canvas canvas, String shape) {
8.         this.canvas = canvas;
9.         this.shape = shape;
10.    }
11.
12.    @Override
13.    public void execute() {
14.        canvas.draw(shape);
15.    }
16.
17.    @Override
18.    public void undo() {
19.        canvas.erase(shape);
20.    }
21.}
```

```
1. // Concrete Command: Erase Command
2. class EraseCommand implements Command {
3.
4.     private Canvas canvas;
5.     private String shape;
6.
7.     public EraseCommand(Canvas canvas, String shape) {
8.         this.canvas = canvas;
9.         this.shape = shape;
10.    }
11.
12.    @Override
13.    public void execute() {
14.        canvas.erase(shape);
15.    }
16.}
```

```

15.     }
16.
17.     @Override
18.     public void undo() {
19.         canvas.draw(shape);
20.     }
21. }

```

Invoker (CommandManager) จัดการการเรียกใช้คำสั่ง execute() รองรับการ undo และ redo

```

1. // Invoker: Command Manager
2. class CommandManager {
3.
4.     private Stack<Command> commandHistory = new Stack<>();
5.     private Stack<Command> redoStack = new Stack<>();
6.
7.     public void executeCommand(Command command) {
8.         command.execute();
9.         commandHistory.push(command);
10.        // Clear redo stack after new command
11.        redoStack.clear();
12.    }
13.
14.    public void undo() {
15.        if (!commandHistory.isEmpty()) {
16.            Command command = commandHistory.pop();
17.            command.undo();
18.            redoStack.push(command);
19.        } else {
20.            System.out.println("Nothing to undo.");
21.        }
22.    }
23.
24.    public void redo() {
25.        if (!redoStack.isEmpty()) {
26.            Command command = redoStack.pop();
27.            command.execute();
28.            commandHistory.push(command);
29.        } else {
30.            System.out.println("Nothing to redo.");
31.        }
32.    }
33.}

```

Client (GraphicEditor) สร้างคำสั่งและส่งให้ CommandManager จัดการ

```

1. // Main Class
2. public class GraphicEditor {
3.     public static void main(String[] args) {
4.
5.         // Receiver
6.         Canvas canvas = new Canvas();
7.
8.         // Commands
9.         Command drawCircle = new DrawCommand(canvas, "Circle");

```

```

10.         Command drawRectangle = new DrawCommand(canvas
11.                                                     , "Rectangle");
12.         Command eraseCircle = new EraseCommand(canvas
13.                                                  , "Circle");
14.
15.         // Invoker
16.         CommandManager commandManager = new CommandManager();
17.
18.         // Execute Commands
19.         // Drawing: Circle
20.         System.out.println("== Drawing Shapes ==");
21.         commandManager.executeCommand(drawCircle);
22.
23.         // Drawing: Rectangle
24.         commandManager.executeCommand(drawRectangle);
25.
26.         System.out.println("\n== Undo ==");
27.         commandManager.undo(); // Erasing: Rectangle
28.         commandManager.undo(); // Erasing: Circle
29.
30.         System.out.println("\n== Redo ==");
31.         commandManager.redo(); // Drawing: Circle
32.         commandManager.redo(); // Drawing: Rectangle
33.
34.         System.out.println("\n== Erasing Circle ==");
35.
36.         // Erasing: Circle
37.         commandManager.executeCommand(eraseCircle);
38.
39.         System.out.println("\n== Undo Erase ==");
40.         commandManager.undo(); // Drawing: Circle
41.     }
42.}

```

ผลลัพธ์

== Drawing Shapes ==

Drawing: Circle

Drawing: Rectangle

== Undo ==

Erasing: Rectangle

Erasing: Circle

== Redo ==

Drawing: Circle

Drawing: Rectangle

== Erasing Circle ==

Erasing: Circle

== Undo Erase ==

Drawing: Circle

**ตัวอย่างที่ 2 ระบบจัดการคำสั่งในเกม (Game Command System)** Command Pattern ในเกมช่วยจัดการคำสั่งของผู้เล่น เช่น การเดิน โจมตี หรือใช้ไอเทม โดยสามารถรองรับ Undo/Redo และช่วยเพิ่มความยืดหยุ่นในการจัดการคำสั่ง เช่น การสร้าง AI ที่ใช้คำสั่งเดียวกับผู้เล่น ในเกมที่ผู้เล่นมีการกระทำต่าง ๆ เช่น เดิน โจมตี หรือใช้ไอเทม การจัดการคำสั่งเหล่านี้ต้อง

- ยืดหยุ่นในการเพิ่มคำสั่งใหม่
- รองรับการ undo (ย้อนกลับการกระทำ)
- จัดการคำสั่งจากผู้เล่นหลายคนในระบบเดียว

การใช้ Command Pattern

- สร้างคำสั่ง เช่น MoveCommand, AttackCommand, UseItemCommand
- เก็บประวัติคำสั่งเพื่อสร้างระบบ replay หรือ undo
- ทำให้ AI หรือผู้เล่นสามารถใช้คำสั่งเดียวกันในระบบเดียว

โครงสร้างระบบ

- Command Interface กำหนดคำสั่ง execute() และ undo()
- Concrete Command คำสั่งเฉพาะ เช่น MoveCommand, AttackCommand
- Receiver ตัวละครในเกมที่ปฏิบัติตามคำสั่ง เช่น Player
- Invoker ตัวที่รับคำสั่งจากผู้เล่น เช่น GameController
- Client ส่วนที่สร้างคำสั่งและกำหนดการทำงานของระบบ

```
1. // Command Interface
2. interface Command {
3.     void execute();
4.     void undo();
5. }
```

Receiver Player เป็นตัวละครที่ปฏิบัติตามคำสั่ง เช่น เดิน โจมตี หรือป้องกัน

```
1. // Receiver: Player
2. class Player {
3.
4.     private String name;
5.
6.     public Player(String name) {
7.         this.name = name;
8.     }
9.
10.    public void move(String direction) {
11.        System.out.println(name + " moves " + direction);
12.    }
13.
14.    public void attack(String target) {
15.        System.out.println(name + " attacks " + target);
16.    }
17.
18.    public void defend() {
19.        System.out.println(name + " defends");
20.    }
21.}
```

Concrete Command (MoveCommand) เป็นคำสั่งให้ตัวละครเคลื่อนที่

```
1. // Concrete Command: Move Command
2. class MoveCommand implements Command {
3.
4.     private Player player;
5.     private String direction;
6.
7.     public MoveCommand(Player player, String direction) {
8.         this.player = player;
9.         this.direction = direction;
10.    }
11.
12.    @Override
13.    public void execute() {
14.        player.move(direction);
15.    }
16.
17.    @Override
18.    public void undo() {
19.        System.out.println("Undo move: " + direction);
20.    }
21.}
```

AttackCommand คำสั่งให้ตัวละครโจมตี

```
1. // Concrete Command: Attack Command
2. class AttackCommand implements Command {
3.
```

```

4.     private Player player;
5.     private String target;
6.
7.     public AttackCommand(Player player, String target) {
8.         this.player = player;
9.         this.target = target;
10.    }
11.
12.    @Override
13.    public void execute() {
14.        player.attack(target);
15.    }
16.
17.    @Override
18.    public void undo() {
19.        System.out.println("Undo attack on: " + target);
20.    }
21.}

```

DefendCommand คำสั่งให้ตัวละครป้องกัน

```

1. // Concrete Command: Defend Command
2. class DefendCommand implements Command {
3.
4.     private Player player;
5.
6.     public DefendCommand(Player player) {
7.         this.player = player;
8.     }
9.
10.    @Override
11.    public void execute() {
12.        player.defend();
13.    }
14.
15.    @Override
16.    public void undo() {
17.        System.out.println("Undo defend");
18.    }
19.}

```

Invoker GameController จัดการการเรียกใช้คำสั่งและรองรับ Undo/Redo

```

1. // Invoker: GameController
2. class GameController {
3.
4.     private Stack<Command> commandHistory = new Stack<>();
5.     private Stack<Command> redoStack = new Stack<>();
6.
7.     public void executeCommand(Command command) {
8.         command.execute();
9.         commandHistory.push(command);
10.        // Clear redo stack after a new command
11.        redoStack.clear();
12.    }
13.}

```

```

14.     public void undo() {
15.         if (!commandHistory.isEmpty()) {
16.             Command command = commandHistory.pop();
17.             command.undo();
18.             redoStack.push(command);
19.         } else {
20.             System.out.println("Nothing to undo.");
21.         }
22.     }
23.
24.     public void redo() {
25.         if (!redoStack.isEmpty()) {
26.             Command command = redoStack.pop();
27.             command.execute();
28.             commandHistory.push(command);
29.         } else {
30.             System.out.println("Nothing to redo.");
31.         }
32.     }
33. }

```

Client สร้างคำสั่งและสั่งให้ GameController ดำเนินการ

```

1. // Main Class
2. public class Game {
3.     public static void main(String[] args) {
4.
5.         // Receiver
6.         Player player = new Player("Hero");
7.
8.         // Commands
9.         Command moveNorth = new MoveCommand(player, "north");
10.        Command attackEnemy = new AttackCommand(player
11.                                                    , "Dragon");
12.        Command defend = new DefendCommand(player);
13.
14.        // Invoker
15.        GameController controller = new GameController();
16.
17.        // Execute Commands
18.        System.out.println("== Player Actions ==");
19.        controller.executeCommand(moveNorth);
20.        controller.executeCommand(attackEnemy);
21.        controller.executeCommand(defend);
22.
23.        System.out.println("\n== Undo Actions ==");
24.        controller.undo(); // Undo defend
25.        controller.undo(); // Undo attack on Dragon
26.
27.        System.out.println("\n== Redo Actions ==");
28.        controller.redo(); // Redo attack on Dragon
29.        controller.redo(); // Hero moves north
30.    }
31. }

```

ผลลัพธ์

== Player Actions ==

Hero moves north

Hero attacks Dragon

Hero defends

== Undo Actions ==

Undo defend

Undo attack on: Dragon

== Redo Actions ==

Hero attacks Dragon

Hero moves north

ข้อดีของการใช้ Command Pattern ในเกม

- Undo/Redo รองรับการย้อนกลับหรือทำซ้ำคำสั่งของผู้เล่น
- ขยายง่าย เพิ่มคำสั่งใหม่ เช่น UseItemCommand หรือ CastSpellCommand ได้ง่าย
- รองรับ AI ใช้คำสั่งเดียวกันในการควบคุม AI
- การบันทึกการเล่น สามารถเก็บลำดับคำสั่งเพื่อสร้างระบบ Replay ได้

### ตัวอย่างที่ 3 ระบบควบคุมอุตสาหกรรม (Industrial Automation) ในโรงงานอุตสาหกรรม

ระบบควบคุมต้องสั่งการอุปกรณ์ต่าง ๆ เช่น

- เริ่มสายพานลำเลียง
- หยุดเครื่องจักร
- เปลี่ยนการตั้งค่าของเครื่องจักร

คำสั่งเหล่านี้มาจากหลายแหล่ง เช่น คอมพิวเตอร์ ปุ่มกด หรือเซ็นเซอร์ และต้องสามารถจัดการคำสั่งให้เป็นระบบ การประยุกต์ใช้ Command Pattern ได้แก่

- สร้างคำสั่ง เช่น StartConveyorCommand, StopMachineCommand, AdjustSettingCommand
- รองรับการเก็บคำสั่งเพื่อการตรวจสอบย้อนหลัง
- เพิ่มความยืดหยุ่นในการเปลี่ยนลำดับการทำงาน

**ตัวอย่างที่ 4 ระบบควบคุมเวิร์กโฟลว์ในองค์กร (Workflow System)** เช่น การอนุมัติเอกสาร ต้องรองรับ การส่งคำขอไปยังผู้อนุมัติหลายคน การย้อนกลับ (undo) หากคำขอถูกยกเลิก การเก็บประวัติคำสั่งเพื่อการตรวจสอบ การประยุกต์ใช้ Command Pattern ได้แก่

- สร้างคำสั่ง เช่น `SubmitRequestCommand`, `ApproveRequestCommand`, `RejectRequestCommand`
- เก็บคำสั่งใน stack เพื่อรองรับ undo/redo
- สร้างระบบ automation สำหรับอนุมัติตามเงื่อนไข

**ตัวอย่างที่ 5 ระบบควบคุมเครื่องจักรในยานยนต์ (Automotive Control System)** ในรถยนต์ยุคใหม่ คำสั่งการควบคุมอุปกรณ์ต่าง ๆ เช่น เปิดไฟหน้า ปรับที่นั่ง เปิด/ปิดหน้าต่าง จะต้องสามารถจัดการคำสั่งเหล่านี้จากปุ่มบนแผงควบคุมหรือแอปพลิเคชันมือถือ การประยุกต์ใช้ Command Pattern ได้แก่

- สร้างคำสั่ง เช่น `TurnOnHeadlightsCommand`, `AdjustSeatCommand`, `OpenWindowCommand`
- รองรับการบันทึกคำสั่งสำหรับปรับค่าที่นั่งหรือการตั้งค่าอื่น ๆ อัตโนมัติ (User Profile)
- เพิ่มความยืดหยุ่นในการเพิ่มคำสั่งใหม่ เช่น เพิ่มคำสั่ง "เปิดหลังคา"

ประโยชน์ของการใช้ Command Pattern ในงานจริง

- แยกหน้าที่ชัดเจน แยกคำสั่งออกจากผู้ส่งคำสั่งและผู้ปฏิบัติงาน
- รองรับ Undo/Redo ช่วยให้สามารถย้อนกลับหรือทำซ้ำคำสั่งได้
- ยืดหยุ่น เพิ่ม/แก้ไขคำสั่งใหม่ได้ง่าย
- การเก็บประวัติคำสั่ง ช่วยในการวิเคราะห์หรือสร้าง workflow อัตโนมัติ
- ลดความซับซ้อน ลดการเขียนโค้ดแบบ if-else หรือ switch-case หลายชั้น